1) Cancer Genomics Dataset

What features are present in the cancer genomics dataset?

The cancer genomics dataset contains the gene expressions of various gene loci in patients with different types of tumors (BRCA, KIRC, COAD, LUAD and PRAD), where each of the ~5500 features corresponds to the expression of a specific gene.

What preprocessing steps did you apply (if any)?

I checked for duplicates, missingness, and non-numeric columns, all of which I found none in the cancer dataset.

What initial observations did you make about the dataset (dimensionality, class balance, missingness, feature scale, etc.)?

The proportions of each type of cancer were as follows: {'BRCA': 300, 'KIRC': 146, 'LUAD': 141, 'PRAD': 136, 'COAD': 78}. This suggests that the dataset is moderately imbalanced, with BRCA having relatively more samples and COAD having relatively fewer.

As for dimensionality, there are 801 samples and 5479 features, indicating that the model has high dimensionality—with nearly seven times more features than samples.

Additionally, the features all had approximately the same scale, as they are all measurements of gene expression ranging from 0.0-20.3 (indicating that standardization may not be necessary). Furthermore, the dataset is multi-class (not binary), with five different cancer types.

2) Model Performance (Heart Disease + Cancer Genomics)

Figure 1: Heart and Cancer Model Metrics

	Heart Model	Cancer Model
accuracy	0.786885	0.981366
precision	0.794117	0.984161
recall	0.818181	0.981366
f1	0.805970	0.981846
roc_auc	0.880411	0.999906

For both datasets, report and discuss model performance using metrics like accuracy, precision, recall, F1-score.

For the heart dataset, I used the best hyperparameters obtained from hyperparameter tuning and achieved the results in Figure 1. These metrics indicate that the model performed somewhat well. The accuracy of 0.78 indicates that the model correctly classifies about 78% of test cases. Precision and recall are both fairly high, indicating that the accuracy and completeness of positive predictions, respectively, were both fairly good; this is further reflected by the F1 score of 0.81. Finally, ROC-AUC was close to 1, which is also an indication of a good classifier. Also, I noted that the cross-validation metrics of the heart disease classifier (see code for full info) were higher than in Figure 1, indicating that the actual performance of the model across many train/test splits may be slightly higher than indicated (e.g. accuracy closer to 0.85).

The cancer dataset metrics are also in Figure 1. They indicate that the cancer classification model performed very well, with accuracy, precision, recall, F1, and ROC-AUC scores upwards of 0.98. This is characteristic of near-perfect classification, with almost all test cases being classified correctly.

Which dataset was easier/harder to classify?

The cancer dataset was easier to classify, as all five of the cancer classification model's metrics were higher than those of the heart disease model.

Which errors were most common (especially in the cancer dataset)?

For the heart disease dataset: judging by the heatmap in Figure 2, it seems that false positives were approximately as common as false negatives (7 vs 6).

The model performed very well on the cancer dataset, with BRCA, KIRC, LUAD, and PRAD having perfect classifications. The only errors were LUAD being misclassified as COAD (twice) and PRAD being misclassified as COAD (once). This suggests that COAD may have some biologically similar features to LUAD and PRAD that may have caused these classifications.

Figure 2: Heart Disease Model Heatmap

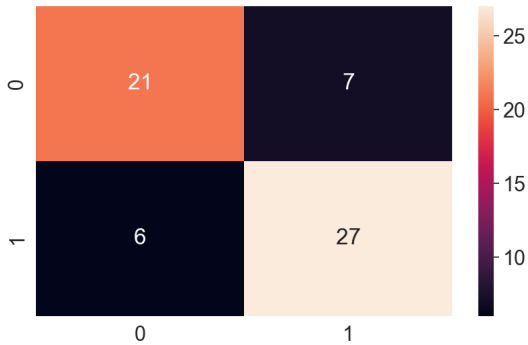


Figure 3: Cancer Model Confusion Matrix

	BRCA	COAD	KIRC	LUAD	PRAD
BRCA	61	0	0	0	0
COAD	0	17	0	0	0
KIRC	0	0	25	0	0
LUAD	0	2	0	27	0
PRAD	0	1	0	0	28

**Note: rows represent true classes and columns represent predicted classes*

3) Hyperparameter Tuning (Gradient Boosting)

Which hyperparameters did you explore (e.g., max_depth ,n_estimators, learning_rate, etc.)? What range of values did you test?

Within the heart model, I explored max_depth of 1, 2, 3; learning_rate of 0.1, 0.05, 0.01; n_estimators of 50, 100, 500, 1000; and subsample of 0.6, 0.8, 1.0.

Within the cancer model, I explored a smaller range of values due to computational limitations. I looked at max_depth of 1, 2; n_estimators of 50, 100; and learning_rate of 0.2, 0.05.

Which configuration performed best on each dataset, and why do you think it worked well?

On the heart dataset, a configuration of {max_depth = 1, learning_rate = 0.1, and n_estimators = 50, subsample = 0.8} worked the best, with the resulting metrics shown above in Figure 1. This combination of hyperparameters may have worked well because it combats overfitting: a low max depth resulted in weak learners; a larger learning rate paired with a smaller number of estimators also combats overfitting; and a small degree of subsampling may also have helped to prevent overfitting without removing too much data.

On the cancer dataset, the best configuration was {max_depth = 1, learning_rate = 0.2, n_estimators = 50}. It is likely that the lower max

Figure 4: Grid Search Table for Heart Disease Classification

learning_rate	max_depth	n_estimators	subsample	mean_test _score
0.10	1	50	0.6	0.905665
0.10	1	50	0.8	0.910040
0.10	1	50	1.0	0.908233
0.10	1	100	0.6	0.903613
0.10	1	100	0.8	0.906200
...	...	...	...	...
0.01	3	500	0.8	0.894335
0.01	3	500	1.0	0.889411
0.01	3	1000	0.6	0.894886
0.01	3	1000	0.8	0.890138
0.01	3	1000	1.0	0.882148

depth, higher learning rate, and lower number of estimators may have worked to prevent overfitting, which is especially relevant on the cancer dataset due to its high dimensionality

Provide grid search table or plot.

See Figure 4, which provides a truncated view of the mean test scores for various hyperparamater configurations (108 total rows; see code for full output).

4) Feature Importance and Interpretation

Discuss which features were most predictive. For gradient boosting: report the top features by importance.

The feature importances in the heart and cancer datasets are shown in Figures 5 and 6, respectively, as ranked by impurity.

In the heart dataset, the features that stand out as having relatively high importance are cp, thal, ca, oldpeak, and slope.

In the cancer dataset, the top twenty genes with the highest importance are shown; it appears these ten features comprise the majority of the importance, with importances becoming ~0 beyond 10 or 20 features. This indicates that, although there are 5000+ features, only a small number of these are actually important in the prediction.

Figure 5: Heart Dataset - Top 10 Features

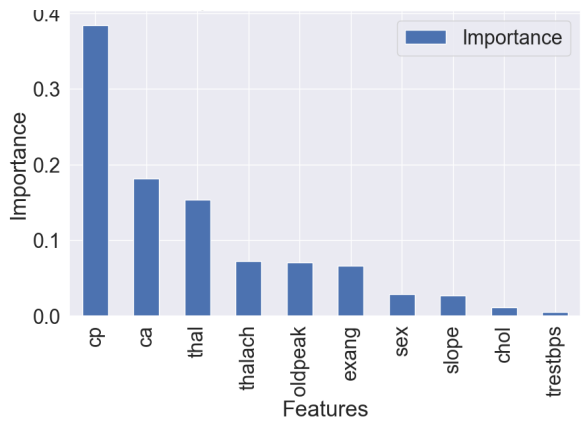
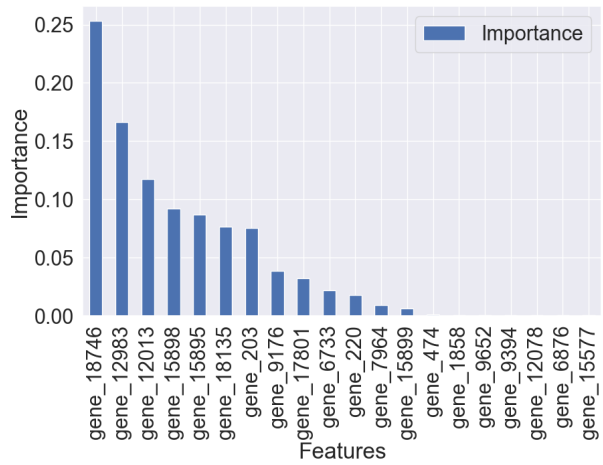


Figure 6: Cancer Dataset - Top 20 Features



Compare feature importance to HW1 feature selection (for example: using Lasso to identify important features). Do the two methods agree on what features matter most? Why or why not?

Comparing the heart disease dataset feature regression coefficients from HW1 (Figure 7) to the gradient boosting feature importances (Figure 5) we can see that there is some (but not complete) overlap between the most important features. For instance, both methods have cp as the top feature, but the gradient boosting method has thal as having high importance and sex as having low importance, while the HW1 method has the opposite (thal with lower importance, sex with high importance).

This is because the two methods measure importance differently; in HW1, we used the regression coefficients of each feature as a measure of importance, where values closer to 1 or -1 indicate stronger linear correlations between the feature and the target variable; on the other hand, in gradient boosting, we use impurity (how much a feature reduces impurity) to measure importance. Thus, one reason for the difference in the top features is that impurity-based feature importance may capture non-linear relationships while regression coefficients do not.

5) Comparison to HW1 - Cancer Genomics Dataset

Did the reduced feature set improve or hurt performance? Provide metrics and a short explanation of what you observed.

The HW1 heart disease logistic regression model was able to achieve 1.0 on all five metrics for the full-feature model. While perfect metrics of 1.0 are unusual, it may be because different cancer types are genetically very distinct, making them easy to classify.

The reduced feature set (with the top features selected based on the gradient boosting feature set, as shown in figure 6) was also able to achieve 1.0 on all five metrics, up until fewer than 10 features were selected (at which point the metrics started to decrease). This supports the observation from earlier that only a few features actually contribute to the final model decision.

I also trained an additional HW1 model using L1 regularization (Lasso) and found that this Lasso model was also able to achieve metrics of 1.0.

Therefore, it appears that all three models (HW1 full-feature, HW1 top-K features, and HW1 Lasso) were able to achieve perfect metrics of 1.0.

Figure 7: Feature Regression Coefficients (HW1)

cp	0.855489
thalach	0.438294
slope	0.353978
restecg	0.191038
fbs	0.053670
age	-0.120732
chol	-0.284776
trestbps	-0.404845
exang	-0.456145
thal	-0.523270
oldpeak	-0.658853
ca	-0.734191
sex	-0.810710

6) Discussion

Discuss limitations of your models and results, including model assumptions and generalizability, potential overfitting (especially with deeper trees or large ensembles), sensitivity to hyperparameters, and data quality issues.

The heart disease model achieved fairly strong performance, but may have had some limitations. For instance, the heart disease dataset had approximately 700 duplicates out of around 1000 total samples. This decreased the number of samples to only 300, which is a relatively small number. For better generalizability and reduced risk of overfitting, it would be better to have a larger number of samples on which to train and test the model.

For the cancer genomics dataset, one potential limitation is that it is prone to overfitting due to its 5000+ features compared to only ~800 samples. However, it seems that the model was still able to perform very well, indicating that overfitting may not have been an issue. Even so, it may be important to train the model on more data to see if it is able to generalize well to external data.

Furthermore, both models are highly sensitive to hyperparameters: changing the max_depth, learning_rate, or num_estimators can hugely affect results, meaning that much of the model's performance depends on its hyperparameters. While the hyperparameter tuning method was meant to combat some of this sensitivity, it is not perfect since only a small number of combinations of hyperparameters were tested by tune_hyperparameters (due to computational power limitations).